

Department of Computer Science and Engineering
Indian Institute of Technology Guwahati

Lab 3 Pre-Lab Reading

Sharing Kernel Information with User Space

A short chapter on protection, shared mappings, and consistent snapshots

CS3106L – Operating Systems: Concepts and Construction
Luit / RISC-V

Read this chapter before starting Graded Lab 3.

Chapter 1

Sharing Kernel Information with User Space

Before you begin

Lab 3 brings together ideas that have already appeared in the course: system calls, page tables, process state, interrupts, and concurrency. The lab itself asks you to complete only a small amount of code, but that code sits at the boundary between the kernel and a user process. A small mistake in this part of an operating system can lead to incorrect data, broken protection, or failures that appear only under timing pressure.

This chapter explains the design before you implement it. It does not give the four graded TODO regions as finished code. Instead, it develops the reasoning that the implementation must follow. Read it once from beginning to end. During the lab, return to the relevant section when a test fails or when you are unsure why a particular synchronization step is needed.

The central question is this: how can the kernel make a few frequently read values available to user space without requiring a system call every time, while still preserving protection and returning a consistent view of the data?

1.1 A Shared Information Page

1.1.1 Why this problem is interesting

A user program often needs small pieces of information that the kernel already knows. A process may want its process ID, the current timer tick count, the number of system calls it has made, or the number of times it has been scheduled. The usual way to ask the kernel for such information is a system call.

For example, a call to `getpid()` eventually causes the processor to execute an `ecall`. Control enters the kernel, the trap handler identifies the system call, the kernel obtains the value, and control later returns to user mode. This is the correct mechanism when the kernel must perform privileged work. Opening a file, creating a process, changing a page table, or writing to a device cannot be replaced by an ordinary user-space load.

However, a read-only value is different. If the kernel already maintains a value and a process reads it very frequently, repeatedly crossing the user-kernel boundary may be unnecessary work. One possible design is to let the kernel maintain a small memory page that user code can read directly.

Lab 3 studies exactly this design. Each process has a small information page. The kernel updates the page. User code reads it. The page is protected so that user code cannot modify it.

At first this may sound like a simple page-table exercise. It is not. The difficult part is not making the page visible. The difficult part is making sure that a user program does not read half of one kernel update and half of another.

1.1.2 A brief system-call refresher

Consider the following user code:

```
int p = getpid();
```

A simplified Luit path is:

```
user getpid()
-> place the system-call number in a7
-> ecall
-> usertrap()
-> syscall()
-> sys_getpid()
-> return through the trap path
-> continue in user mode
```

This path does more work than an ordinary memory load. The lab therefore introduces a second path for a small set of read-mostly information. The ordinary system calls remain valid; the shared page is an additional fast path, not a replacement for the system-call mechanism.

1.1.3 One virtual address, one page per process

The Lab 3 starter maps the information page at a fixed user virtual address:

```
USYSINFO_VA = 0x7FFF000
```

Every process uses this same virtual address. That does not mean every process sees the same physical page. The page table of each process translates the virtual address independently.

For example:

Process A	Process B
VA 0x7FFF000	VA 0x7FFF000
v	v
physical page PA	physical page PB

When process A is running, its page table maps 0x7FFF000 to page PA. When process B is running, its page table maps the same virtual address to page PB. This is ordinary virtual-memory isolation applied to a special-purpose page.

The kernel also keeps a kernel-side pointer to the backing page for each process. The kernel therefore has a convenient way to update the data, while the user process reaches the same data through its user virtual address.

1.1.4 The permissions on the mapping

The user mapping is created with read and user permissions, but without write permission. In RISC-V page-table terms, the important flags are:

Flag	Meaning in this lab
PTE_U	User mode may access the page.
PTE_R	Loads from the page are allowed.
PTE_W absent	Stores from user mode are not allowed.

Thus user code can read the information page, but a user store to the same virtual address should cause a page fault. The kernel remains the owner of the data.

It is important to separate this protection property from the consistency problem studied later in the chapter. Making a page read-only prevents the user process from changing it. It does not prevent the kernel from changing the page while the user process is reading it. A page can therefore be perfectly protected and still return an inconsistent multi-field snapshot.

1.2 The Data Shared Across the Boundary

1.2.1 The shared structure

The interface between the kernel and user code is defined in `kernel/usysinfo.h`. The structure used by the current lab has the following form:

```
struct usysinfo {
    uint32 version;
    volatile uint32 seq;
    int pid;
    int hart;
    uint64 ticks;
    uint64 syscall_count;
    uint64 ctxsw_count;
    uint32 state_gen;
};
```

The fields are deliberately small and simple. They are enough to study the design without turning the lab into a large monitoring system.

Field	Meaning
<code>version</code>	Version of the kernel-user layout.
<code>seq</code>	Sequence counter used while publishing and reading a snapshot.
<code>pid</code>	Process ID.
<code>hart</code>	RISC-V hart associated with the most recent publication.
<code>ticks</code>	Kernel tick value at the time of publication.
<code>syscall_count</code>	Number of valid system calls counted for the process.
<code>ctxsw_count</code>	Number of scheduler dispatches counted for the process.
<code>state_gen</code>	A generation value advanced with the supplied context-switch counter.

1.2.2 Why the structure contains a version

The layout of a shared page is an agreement between kernel code and user code. If one side changes the layout while the other side still expects an older layout, the same bytes may be interpreted differently. This is an ABI problem.

For the current lab, the interface version is `USYSINFO_VERSION = 1`. The user library can check this field before interpreting the rest of the page. A later kernel could introduce a different version if the structure changes.

This is a small example of a general operating-systems idea: once information crosses a protection boundary, the layout and meaning of that information become an interface, not merely an internal C structure.

1.2.3 Where the values are updated

The Lab 3 starter already contains the lifecycle and accounting hooks needed by the assignment. You are not being asked to invent new counters or search the kernel for arbitrary places to update them.

One publication occurs after a valid system call. Conceptually, the kernel completes the system call, updates the process's system-call accounting, and then refreshes the shared information page. Another publication occurs when the scheduler dispatches a process. The supplied scheduler updates the context-switch-related fields and then refreshes the shared page before switching into the process.

These two paths are important later because they show that the writer can be reached from more than one kernel context. The consistency protocol must be correct for both.

1.3 The Consistency Problem

1.3.1 Why a normal structure copy can fail

Suppose the kernel wants to publish two related values. For simplicity, consider only `ctxsw_count` and `state_gen`. Assume both currently contain 10. During the next scheduler dispatch, the kernel intends to publish 11 in both fields.

A possible timing is shown below.

Time	Kernel writer	User reader
1	writes <code>ctxsw_count</code> = 11	
2		reads <code>ctxsw_count</code> = 11
3		reads <code>state_gen</code> = 10
4	writes <code>state_gen</code> = 11	

The user has now observed the pair (11, 10). The kernel did not intend that pair to represent a complete state. One field came from the new publication and the other came from the old publication. This is a *torn snapshot*.

The problem becomes more likely to matter as the structure grows, as updates become more frequent, or as execution moves across multiple harts. A direct memory mapping therefore needs a consistency protocol in addition to page-table protection.

1.3.2 Using an odd/even sequence counter

Lab 3 uses the field `seq` as a sequence counter. The protocol gives odd and even values different meanings:

Even value	No publication is currently in progress.
Odd value	A writer is currently changing the page.

Suppose a stable page has `seq` = 40. Before changing the data fields, the writer moves the counter to an odd value, such as 41. After the complete new state has been published, the writer moves the counter to the next even value, such as 42.

The reader uses this information to decide whether a copy can be trusted. It observes the sequence value before copying the fields and again after the copy. The copy is accepted only when the two observed values are the same and even.

There are three useful cases to understand.

No writer overlaps the reader

The page is stable at sequence 40. The reader observes 40, copies the fields, and observes 40 again. Because the values are equal and even, the copy is accepted.

```
first sequence value = 40
copy the fields
second sequence value = 40
result: accept
```

A writer runs during the copy

The reader first observes 40. While it is copying fields, the writer changes the page and completes a new publication. The reader finally observes 42.

```
first sequence value = 40
copy begins
writer publishes a new state: 40 -> 41 -> 42
copy ends
second sequence value = 42
result: retry
```

The reader does not need to know which individual field was old or new. The changed sequence value is enough to reject the whole copy.

The reader arrives while a writer is active

If the first sequence value is odd, for example 41, the reader already knows that the page is being changed. It simply retries instead of copying data that is known to be unstable.

This style of reader is lock-free from user space. The user process does not acquire a kernel spinlock and does not make a system call merely to protect the read. It keeps trying until it sees a stable publication.

1.4 Memory Ordering and Writer Serialization**1.4.1 Why source-code order is not enough**

The odd/even protocol depends on an ordering guarantee. The writer conceptually performs three stages:

```
mark the page as being updated
publish the new field values
mark the page as stable again
```

For the protocol to work, readers must not observe the final even sequence value before the new fields are visible. Similarly, the writer must not make some new fields visible before readers can observe that the sequence has become odd.

C source-code order alone is not the synchronization guarantee we need. Compilers and processors may reorder memory operations when such reordering is legal for ordinary single-threaded execution. Synchronization code therefore uses explicit ordering operations.

The lab uses `__sync_synchronize()` as a full memory barrier. In conceptual form, the writer has the following shape:

```
make seq odd
```

```
memory barrier
publish all data fields
memory barrier
make seq even
```

The reader also uses barriers around the field-copy portion so that its two sequence observations surround the data copy in the intended order.

For this lab, you do not need to redesign the RISC-V memory model or replace the required barrier primitive with another mechanism. You do need to understand why the barriers belong to the protocol.

1.4.2 A second problem: two writers must not overlap

A sequence-counter reader can detect that a writer is active. However, this simple protocol assumes that writers themselves are serialized. Two writers must not update the same page at the same time.

This point matters in Luit because `usysinfo_update()` can be reached from different kernel paths. One call occurs after a system call. Another occurs from the scheduler. During the system-call path, interrupts may already have been enabled. A timer interrupt can therefore arrive while the publication code is running.

Imagine the following sequence of events. A publication begins and makes `seq` odd. Before it finishes, a timer interrupt causes the running process to yield. Later, the scheduler dispatches the same process and tries to publish the page again. The second writer is now entering while the first publication was incomplete. The odd/even protocol was not designed for that situation.

The publication operation is small, so Luit prevents this kind of interrupt-driven writer re-entry by using `push_off()` and `pop_off()` around the writer's critical section.

1.4.3 Why Luit uses `push_off()` and `pop_off()`

A direct pair of `intr_off()` and `intr_on()` would be unsafe when the caller already had interrupts disabled. Calling `intr_on()` at the end would incorrectly enable interrupts that the caller expected to remain disabled.

`push_off()` and `pop_off()` are nest-aware. They remember whether interrupts were originally enabled and maintain a nesting depth. Interrupts are restored only when the final matching `pop_off()` is reached and only if they were enabled before the outermost `push_off()`.

The writer therefore needs two different forms of protection for two different reasons. The sequence counter and memory barriers protect readers from partial publication. The interrupt-control pair prevents overlapping publication paths on the same hart from breaking the writer protocol.

1.5 The User-Side Read Path

1.5.1 Taking a stable snapshot

The user library needs a function that copies the shared information into an ordinary user-space destination structure. Its design follows directly from the earlier discussion.

Before using the shared layout, the reader checks that the destination is usable and that the page reports an ABI version it understands. It then repeatedly performs a read attempt. Each attempt observes the first sequence value. An odd value causes an immediate retry. Otherwise the reader copies the fields and observes the sequence value again. The copy is kept only when the two sequence values are equal and even.

Written as algorithmic steps rather than implementation code, the reader is:

1. Check that the caller supplied a valid destination.
2. Check that the shared page uses the expected interface version.

3. Read the sequence counter.
4. If the counter is odd, retry the attempt.
5. Copy the shared fields into the caller's destination, using the required ordering barriers around the copy.
6. Read the sequence counter again.
7. Accept the snapshot only if the two sequence values are equal and even; otherwise retry.

Once this function returns successfully, the caller owns an ordinary copy. Later kernel updates change the shared page, not the already returned snapshot.

1.5.2 Why the snapshot function copies data

It may seem simpler to return a pointer to the shared page and let callers read individual fields directly. That would lose the main consistency guarantee. One field could be read before a kernel update and another field after it.

A validated copy defines a clear boundary: all fields in the returned structure belong to one accepted observation of the shared page.

1.5.3 Fast accessors

After a correct snapshot operation exists, functions such as `u_getpid()` and `u_uptime()` can use it. Conceptually, `u_getpid()` obtains a stable snapshot and returns the `pid` field; `u_uptime()` does the same for `ticks`.

On the normal compatible path, these operations should not execute `ecall`. That is the performance idea being studied. The lab retains the ordinary system calls as a fallback when the shared-page interface cannot be used. This is a common systems pattern: use a fast path when its assumptions hold, but retain a slower general mechanism for exceptional cases.

The public test verifies this property indirectly by checking that normal calls to the fast accessors do not increase the process's system-call counter.

1.6 Lifecycle of the Special Mapping

1.6.1 Creation, exec, and exit

The Lab 3 starter already handles the lifetime of the information page. This code is supplied, but it is worth understanding because special mappings must be managed throughout the life of a process.

When a process is created, Luit allocates a physical information page and maps it at `USYSINFO_VA`. Each process receives its own backing page.

An `exec()` call replaces the process's ordinary user address space with a new one. A special mapping that existed only in the old page table would disappear unless the kernel deliberately places it in the new page table. The supplied Lab 3 code remaps the process's information page while constructing the new image.

When the process exits, the special page must be unmapped and its physical memory must be freed. This requires explicit handling because the page lies outside the ordinary `[0, p->sz)` region that generic user-memory cleanup normally processes.

1.6.2 Why `sbrk()` must respect the reserved address

The fixed information-page address is reserved for the kernel-user interface. Ordinary process memory must not grow into it. The starter therefore prevents `sbrk()` from extending the normal process image into `USYSINFO_VA`.

This is a useful general lesson. Adding one special page affects more than the code that first creates the mapping. The design must consider allocation, address-space growth, `exec()`, and process destruction as a single lifecycle.

1.7 Reading the Tests as Specifications

1.7.1 What each public check means

A good way to debug an operating-systems lab is to translate each test result into a property of the design. The Lab 3 tests are not independent tricks; they follow directly from the concepts in this chapter.

Observed test	Property being checked
Snapshot returns the correct PID	Kernel and user agree on the mapping and structure layout.
Large snapshot stress reports no torn pair	The publication and retry protocol gives a consistent multi-field view.
Fast PID does not increase syscall count	The normal <code>u_getpid()</code> path avoids <code>ecall</code> .
Fast uptime does not increase syscall count	The same is true for another shared value.
A user store to the page faults	The mapping is user-readable but not user-writable.
Parent and child report their own PIDs	The page is per-process rather than accidentally global.
The interface still works after <code>exec()</code>	The special page is present in the new page table.
The test also runs with several harts	The solution is not correct only under the simplest timing.

The normal Luit regression tests must continue to pass as well. A kernel feature is not correct if it implements the new behaviour by breaking existing behaviour.

1.7.2 Common mistakes and the idea behind each one

Several tempting implementations return correct values in simple runs but violate the design.

A read-only mapping does not remove the need for a sequence protocol. Read-only permission controls user writes; it does not stop concurrent kernel writes.

Checking `seq` only before the copy is not enough. A writer can begin immediately after that check. The reader needs a second observation after the copy.

Equal sequence values are not sufficient if the value is odd. An odd value means a publication was in progress.

The barriers are not decorative. Removing them weakens the ordering guarantee on which the protocol depends.

The sequence counter does not make multiple simultaneous writers safe. The short writer path must still be serialized against interrupt-driven re-entry.

Finally, implementing `u_getpid()` by always calling the ordinary `getpid()` system call may produce the correct number, but it does not implement the fast path that the lab is designed to study.

1.8 Preparing to Implement Lab 3

1.8.1 Files worth reading first

Before editing the TODO regions, trace the relevant code once. The following files are the most useful starting points:

kernel/usysinfo.h The fixed virtual address, interface version, and shared structure.

kernel/usysinfo.c The supplied mapping and lifecycle support, together with the kernel publication TODO.

kernel/syscall.c A call site that publishes the page after system-call accounting.

kernel/proc.c The scheduler call site that publishes the page around dispatch accounting.

kernel/spinlock.c The implementation of `push_off()` and `pop_off()`.

user/ulib.c The user-side snapshot and fast-accessor TODO regions.

user/usitest.c The public checks and the properties they expect.

A useful data-flow picture is:

```
process state
-> kernel publishes one stable state
-> per-process information page
-> user validates one snapshot
-> u_getpid() / u_uptime()
```

The implementation is small once this flow is clear.

1.8.2 Questions to answer before the lab

You should be able to answer the following questions without referring to finished solution code.

1. Why can two processes use the same `USYSINFO_VA` and still see different information pages?
2. Why is the page mapped with user-read permission but without user-write permission?
3. What is a torn snapshot? Give an example involving two fields.
4. What does an odd sequence value mean? What does an even sequence value mean?
5. Why does the reader inspect the sequence counter both before and after copying the fields?
6. What ordering problem do the memory barriers prevent?
7. Why must the writer be protected from interrupt-driven re-entry?
8. Why are `push_off()` and `pop_off()` preferable to a simple `intr_off()` / `intr_on()` pair here?
9. Why does a correct fast `u_getpid()` normally avoid `ecall`?
10. Why does the special page need explicit handling during `exec()` and process cleanup?

1.9 Summary

Lab 3 uses virtual memory to make a small kernel-maintained page visible in user space. Page-table permissions allow the process to read the page but prevent it from changing kernel-owned information. Because the kernel may update several fields while user code is reading them, protection alone is not enough. The lab therefore uses an odd/even sequence counter, memory-ordering barriers, and a retrying user reader to obtain one consistent snapshot.

The writer has one additional responsibility in Luit: the publication path must not be re-entered by another writer before it finishes. The short publication operation therefore uses the kernel's nest-aware interrupt-control mechanism. Once a stable snapshot operation exists, small user helpers such as `u_getpid()` and `u_uptime()` can read kernel-published state without a system call on the normal path.

The assignment is therefore not mainly about copying fields into a structure. It is about designing a small, protected, concurrent interface across the kernel-user boundary.